

PRACTICE 4: MicroServices

Date: 14-agosto-2025



Professor: Cesar Eduardo Inda Ceniceros

Students:

- Hugo Misael Caro Jacobo
- Jessie Alain Peña Ramirez
- Jesus Eduardo Romero Isiordia
- Angel Michel Vela Flores
- Elvis Vergara Hernandez

Team: La Gestapo

Introduction

In this practice, a system based on microservices architecture was developed for a rotisserie business, integrating a REST API built with FastAPI and an independent console interaction module. Each component operates autonomously, communicating through well-defined interfaces, which facilitates maintenance, scalability, and reusability.

The main microservice, implemented with FastAPI, exposes endpoints to retrieve the product menu and register users with specific validations, while the console microservice manages direct interaction with the user, allowing login, registration, and price visualization. Additionally, the NetworkX library was incorporated to generate graphical representations of the system's flow and different types of graphs, thus enhancing the understanding of communication logic between modules.

Requirements

Before starting the development of the microservices, it was necessary to install the dependencies required for running the FastAPI framework and its development server, Uvicorn. FastAPI was chosen for its speed, simplicity, and compatibility with Python type hints, while Uvicorn was used as the ASGI server to run the application. The installation was carried out using Python's package manager with the following command:

```
pip install fastapi uvicorn
```

Development

For the development phase, the implementation focused on creating the api.py file, which contains the FastAPI microservice responsible for managing the menu retrieval and user registration functionalities. This file defines the application instance, the available endpoints, the data validation rules, and the response formats. The following images show the complete source code of the api.py file, illustrating the structure and logic used in its implementation.

```
#todo: Correrlo en consola: uvicorn api:app --reload

#todo: en el navegador: http://127.0.0.1:8000/menu
#todo:API: http://127.0.0.1:8000
#todo: Docs interactivas: http://127.0.0.1:8000/docs

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from App_Scorpionsfood import validar_email

app = FastAPI(title="ScorpionsFood API", version="2.0")

# Re-declaramos el menú aquí para NO modificar el base.
MENU_ITEMS = [
    "Escorpión al chocolate 🍫🐍",
    "Escorpión a la mexicana 🌮🌶️🐍",
    "Escorpión a la KFC 🍗🔥🐍",
]

class Registro(BaseModel):
    nombre: str = Field(min_length=1, description="Nombre del usuario")
    email: str = Field(min_length=5, description="Correo electrónico a validar")

@app.get("/menu")
def get_menu():
    return {"menu": MENU_ITEMS}

@app.post("/registro")
def post_registro(data: Registro):
    if not validar_email(data.email):
        raise HTTPException(status_code=400, detail="Correo electrónico inválido")

    #!Aquí podrías guardar en una BD; por ahora devolvemos OK.
    return {"status": "ok", "mensaje": f"Usuario '{data.nombre}' registrado con éxito."}
```

RESULTS

Once the api.py file was implemented, the FastAPI microservice was executed using Uvicorn with the command:

```
uvicorn api:app --reload
```

Upon running the API, the development server started successfully, displaying information similar to the following in the terminal:

```
INFO: Will watch for changes in these directories: ['<project_path>']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [PID]
INFO: Started server process [PID]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Accessing <http://127.0.0.1:8000/menu> returned a JSON response with the available menu items:

```
{
  "menu": [
    "Pollo entero 🍗 ",
    "1/2 Pollo 🍗 ",
    "1/4 Pollo 🍗 ",
    "Papas fritas 🍟 ",
    "Gaseosa 🥤 "
  ]
}
```

Issues

During the development and execution of the microservice, some common issues were encountered. The first was related to installing the required dependencies for FastAPI and Uvicorn, which caused errors until the correct Python environment and package versions were set up. The second issue was that the file needed to run the Uvicorn command (api.py) was sometimes outside the directory from which the command was executed, preventing the server from starting. These issues were resolved by ensuring that the dependencies were properly installed and that the terminal was pointed to the correct project folder containing the api.py file.

Conclusion

In this practice, a microservices-based system for a rotisserie was successfully implemented, combining a FastAPI REST API and a console interaction module. The development process allowed for the understanding and application of core concepts of microservices, such as modularity, separation of concerns, and independent deployment of components.

Through this practice, the team gained experience in setting up and validating endpoints, handling user input, and visualizing system flows with NetworkX graphs. Additionally, common issues related to dependency management and project structure were identified and resolved, reinforcing good practices for future microservice development.

Overall, this exercise demonstrated the advantages of using microservices for building scalable, maintainable, and interactive applications, while also providing practical experience in Python programming, API design, and graphical representation of system logic.